What Should You Test in a Model Test?

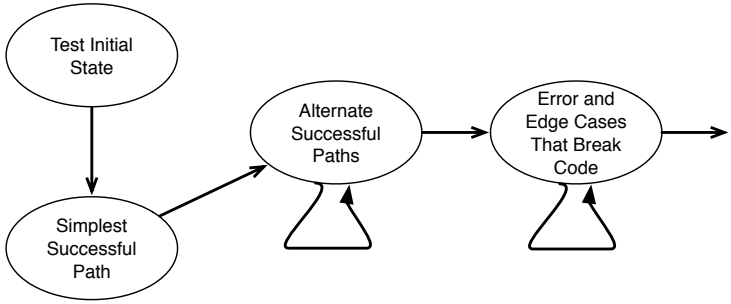
Models. Next question?

Okay, Funny Man, What Makes a Good Set of Model Tests?

There are a lot of different, sometimes conflicting, goals for tests. It's hard to know where to start your TDD process, how many tests to write, and when you're done. The classic description of the process—"write a simple test, make it pass, then refactor"—doesn't provide a lot of affirmative guidance or direction for the larger process of adding a feature.

A TDD Metaprocess

The following diagram illustrates a metaprocess that reflects how I write a new business-logic feature. (I handle view logic a little differently.) This process is more of a guideline than a hard-and-fast checklist. As the logic gets more complex, and the less I know about the implementation when I start, the closer I stick to this process and the smaller the steps I take.



Often the best place to start is with a test that describes an initial state of the system, without invoking logic. This is especially useful if you're test-driving a new class, so the first test just sets up the class and verifies the initial state of instance variables. If you're working in an existing class, this step may not be necessary.

Next, determine the main cases that drive the new logic. Sometimes there will be only one primary case, like "calculate the total value of a user's purchase." Sometimes there will be many; "calculate tax on a purchase" might have lots of cases based on the user's location or the items being purchased.

Take that list and write tests for the main cases one at a time. I do not recommend writing multiple failing tests simultaneously—that turns out to be confusing. It can be helpful to use comments or pending tests to at least note